Operating Systems - Assignment 10 Report

xv6 (x86, MIT 6.828 xv6-public) | Topics: physical memory tracking, virtual-to-physical translation, page-table management and page replacement

Question 1 - Tracking Physical Memory Allocation

Implementation. In `kalloc.c` two counters were added: `kpage_allocs` (incremented on every successful `kalloc()`) and `kpage_frees` (incremented on every `kfree()`). Both are skipped during allocator initialisation (`kmem.use_lock == 0`) so the counters reflect real allocation activity after boot, and the free-side increment happens inside the allocator lock. A helper `kfree_pages()` walks the free list to return the exact number of available frames. The new system call `memstats()` (`sysproc.c`, number 22) prints total allocated pages, total freed pages, currently allocated pages (allocs - frees) and available pages (exact free-list length), with the total managed frame count `(PHYSTOP - V2P(end))/PGSIZE` for context.

Test (memtest.c). Prints stats, allocates 10 pages with `sbrk()` and touches them, prints stats again, then releases the pages (`sbrk(-40960)`) and prints stats a third time.

Observation. After allocation: total allocated rose 425 → 435 and available fell 56789 → 56779 (exactly 10 frames). After release: total freed rose 214 → 224 and available returned to 56789. The counters track allocation and deallocation precisely.

Question 2 - Virtual-to-Physical Address Translation

Implementation. The system call `vtop(int va)` (number 23) is backed by `vtop()` in `vm.c`. It calls `walkpgdir(myproc()->pgdir, va, 0)` (no allocation of missing tables), rejects the address when the entry is missing or `PTE_P` is clear (returns -1), and otherwise returns `PTE_ADDR(*pte) | (va & 0xFFF)` - the physical frame address combined with the page offset.

Test (vttest.c). Allocates 4 pages with `sbrk()`, translates three addresses (block start, second page, an offset inside a page) and then the unmapped address `0x40000000`.

Observation. e.g. `va 0x39C4 → pa 0xDEE39C4`: the low 12 bits (offset `0x9C4`) are preserved, only the frame is translated. The unmapped address returned -1 ("not mapped"), showing the PTE_P validity check works.

Question 3 - Page Table Management and Page Protection

Implementation. Two system calls in `vm.c`: `pageinfo(va)` (number 24) locates the PTE and prints the physical frame number (`PTE_ADDR(*pte) >> 12`), full physical address and the P / W / U flags; `setreadonly(va)` (number 25) clears `PTE_W` in the entry and then reloads `cr3` (`lcr3(V2P(myproc()->pgdir))`) to flush the TLB so the new permission takes effect immediately.

Test (pagetest.c). Allocates one page, prints its info (flags `P W U`), makes it read-only, prints info again (flags `P U` - W cleared), then writes to the page.

Observation. The write faults: the kernel prints `pid 5 pagetest: trap 14 err 7 ... addr 0x3000--kill proc`. Error code 7 = protection violation (bit 0) caused by a write (bit 1) in user mode (bit 2). The page fault confirms that clearing PTE_W genuinely removes write permission.

Question 4 - Page Replacement: FIFO, LRU and Optimal

Implementation. `pagereplace.c` simulates all three algorithms in user space on the reference string `7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1` (20 references). FIFO keeps a circular queue and evicts the oldest-loaded page; LRU records the time of each page's last reference and evicts the least recently used; Optimal looks ahead in the reference string and evicts the page whose next use is farthest away (or never). Every reference prints the frame contents and HIT/FAULT.

Observed results (all computed by the program running inside xv6):

Frames	Algorithm	Faults	Hits	Hit ratio	Fault ratio
3	FIFO	15	5	25%	75%
3	LRU	12	8	40%	60%
3	Optimal	9	11	55%	45%
4	FIFO	10	10	50%	50%
4	LRU	8	12	60%	40%
4	Optimal	8	12	60%	40%
5	FIFO	9	11	55%	45%
5	LRU	7	13	65%	35%
5	Optimal	7	13	65%	35%

Conclusion. Optimal performs best: 9 faults with 3 frames, matching the theoretical minimum because it exploits future knowledge of the reference string. LRU is a close second (12 at 3 frames, tying Optimal at 4 and 5 frames) since this string has strong temporal locality - recently used pages (0, 1, 2) tend to be reused soon. FIFO is worst at every frame count because it ignores usage and can evict a frequently reused page (e.g. page 0). Fault counts fall as frames increase for all three algorithms, and no Belady's anomaly appears for FIFO on this string.

Files and modifications

- **Kernel:** `kalloc.c` (Q1 counters + `kfree_pages()`), `vm.c` (`vtop()`, `pageinfo()`, `setreadonly()`), `sysproc.c` (syscall handlers), `syscall.c`/`syscall.h` (table + numbers), `user.h`/`usys.S` (user stubs), `defs.h` (declarations). Every change is commented `Assignment 10, Qx`.
- **User programs:** `memtest.c`, `vttest.c`, `pagetest.c`, `pagereplace.c` (added to `UPROGS` in the Makefile).
- **Evidence:** `screenshots/` (real QEMU VGA screen captures of each run) and `evidence/terminal-output.txt` (complete serial log of one boot running all four tests).